
NetScale

A High-Throughput Parallel Network Packet Visualizer & Analytics Dashboard

Submitted by:

FajarShahzad 2023-CS-07

Supervised by:

Sir Waqas Ali Zafar

Course:

Parallel and Distributed Computing Lab

Project Type:

Comprehensive Technical Specification & Project Proposal

System Architecture:

Multi-Stage Producer-Consumer Pipeline with Lock-Free Concurrent Ring Buffers

Language / Stack:

C · OpenMP 5.0

Target Throughput:

1,000,000 packets/second · 60 FPS UI · 0 Packet Drops

Department of Computer Science

**University of Engineering and Technology Lahore,
Pakistan**

1. Introduction & Large-Scale Domain Mechanics

At a large scale, modern computer networks transmit data broken down into discrete chunks called **Packets**. In high-speed network environments — such as a Gigabit Ethernet (1 Gbps) line or when processing a massive, multi-gigabyte Network Capture File (.pcap) — data floods the Operating System's network interface card (NIC) ring buffer at an intense rate.

The Math of Network Saturation

```
A 1 Gbps connection running at full capacity --> 125,000,000 bytes / second

Average packet size (mixed traffic, DNS, ACKs) --> 125 bytes

# Derived throughput requirement:

Required CPU throughput --> 1,000,000 packets / second

Time budget per packet --> 1 microsecond (us)

# The critical threshold:

Processing at 1.1 us/packet --> BUFFER SATURATION & DATA LOSS
```

If the processing pipeline takes even **1.1 microseconds per packet**, the input buffer will saturate, causing a massive, unrecoverable data backlog with zero possibility of recovery on a single-threaded system.

2. Small-Scale Data Structure: Deep Packet Anatomy

At the lowest level of execution, a network packet arrives as a raw, unstructured stream of contiguous binary bytes. To visualize it, the CPU must perform rapid **pointer-arithmetic slicing** to decode the nested protocol headers.

RAW PACKET BYTES		
Ethernet Header (14 Bytes)	IP Header (Minimum 20 Bytes)	TCP / UDP / ICMP Header (Variable: 8–20+ Bytes)
Layer 2 — Data Link	Layer 3 — Network	Layer 4 — Transport

Figure 1 — Raw Packet Byte Layout Across OSI Layers 2–4

The Parsing Operations

- Layer 2 — Data Link Layer:** The parser extracts the Ethernet Frame Header (14 bytes) to determine Source and Destination MAC addresses. It checks the EtherType field (2 bytes) to verify whether the payload is IPv4 or IPv6.
- Layer 3 — Network Layer:** The parser shifts its memory offset by 14 bytes to parse the

IP Header. It reads the 4-bit Version, the Header Length, and extracts the 32-bit Source IP and 32-bit Destination IP. It also reads the Protocol byte to route the packet forward (6 = TCP, 17 = UDP).

- **Layer 4 — Transport Layer:** The parser jumps past the IP header to read the Transport Layer Header. It isolates the Source Port (2 bytes) and Destination Port (2 bytes) to identify the application: Port 80 (HTTP), Port 443 (HTTPS), Port 53 (DNS).

3. The Sequential Reality: Why It Fails — "The Packet-Drop Wall"

In a standard single-threaded program, a single CPU core is responsible for the entire operation synchronously inside one rigid loop:

```
[Network Interface] => [Capture Bytes] => [Parse Headers] => [Update Hash Tables] => [Render Graphics]
```

The Fatal Mechanics of the Sequential Bottleneck

- **The Synchronous Block:** When the single thread finishes parsing a packet, it cannot immediately capture the next one. It is forced to pause and execute the full GUI rendering pipeline (drawing lines, updating graphs, recalculating coordinates).
- **The Render Lag:** Redrawing a GUI at 60 FPS takes roughly **16.6 ms** per frame. While the single thread is busy rendering pixels, **16,600 new packets** have already arrived at the network interface.
- **Buffer Overflow:** The OS kernel's network ring buffer reaches 100% capacity. The OS is forced to drop all incoming packets and the dashboard's analytics become completely inaccurate.
- **UI Hard Freeze:** Loading a large 2 GB .pcap file causes the single thread to loop through millions of rows, causing the OS to mark the window as *"Not Responding"*.

4. The Parallel Solution: Multi-Threaded Pipeline Architecture

To handle high-throughput traffic without dropping data, this project splits the synchronous execution loop into an asynchronous, decoupled **Producer-Consumer Thread Pipeline**.

```
+      +

| Network Interface Card |

| or Heavy PCAP File |

+-----+      +

| (Raw

Bytes) v

+-----+      +

| THREAD 1: Capture Daemon | [PRODUCER]

| (Zero Analytics / Fast) |

+-----+      +

| (Enqueues

Pointer) v

+-----+      +
```

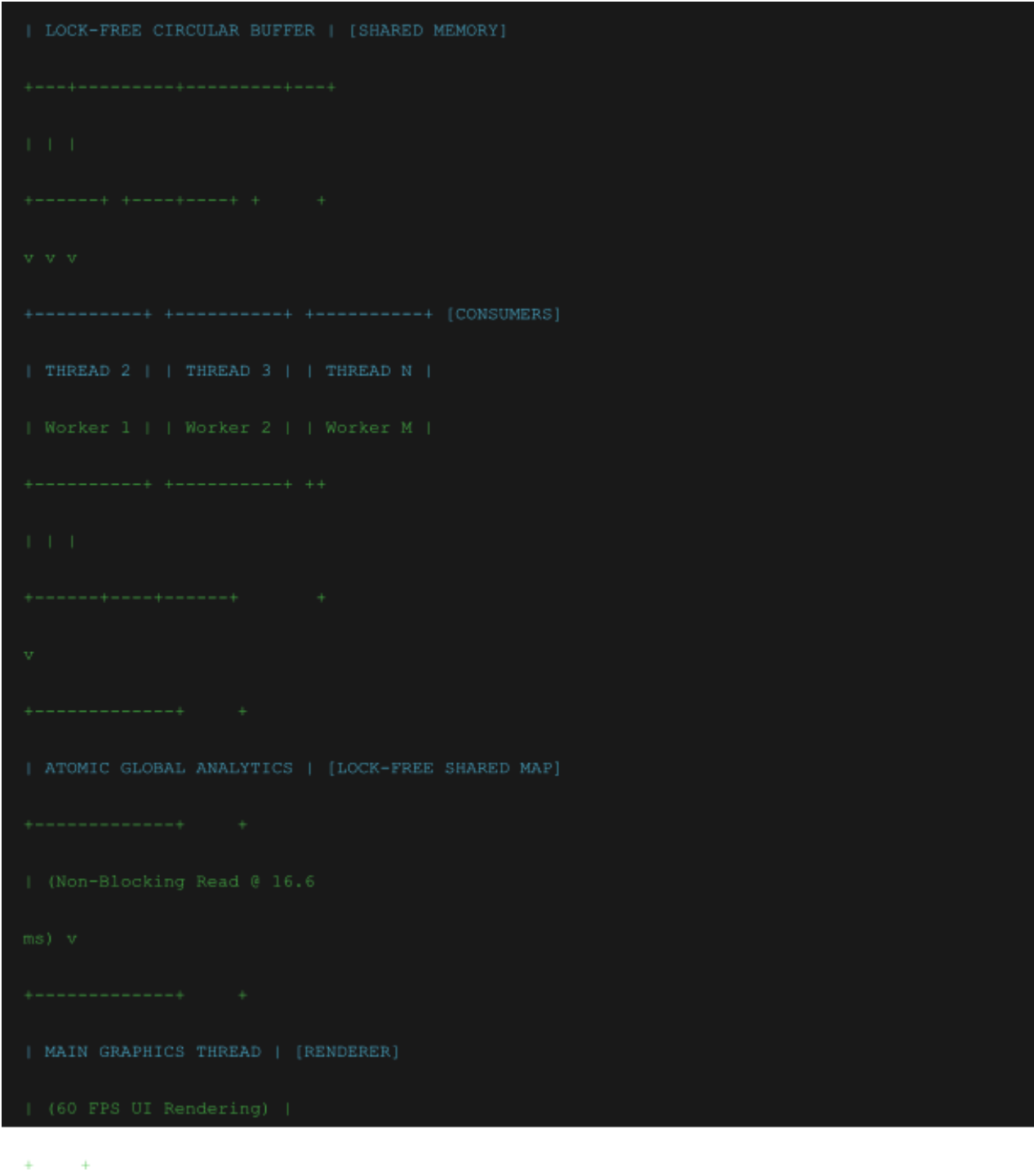


Figure 2 — Multi-Threaded Producer-Consumer Pipeline Architecture

Detailed Component Breakdown

Layer 1 — Capture Daemon (Thread 1 · The Producer)

This thread is pinned to a dedicated CPU core. It performs absolutely no math, no parsing, and no logging. Its sole responsibility is to fetch raw bytes from the network socket or .pcap file using memory-mapped I/O (mmap). As soon as a packet is pulled, it is written into a pre-allocated Lock-Free Circular Ring Buffer and the write-pointer is incremented. This thread completes its loop in nanoseconds, guaranteeing zero packet drops.

Layer 2 — Parser Worker Pool (Threads 2–N · The Consumers)

A pool of hardware threads managed via OpenMP Tasks or Pthreads monitors the circular buffer's read-pointer. Data Parallelism: Thread 2 grabs Packet #1, Thread 3 grabs Packet #2, and Thread 4 grabs Packet #3 simultaneously. Once parsing is complete, each worker aggregates its metrics into the global analytics database using lock-free atomic operations (`std::atomic` / `#pragma omp atomic`), ensuring no threads stall each other.

Layer 3 — Independent UI Renderer (Main Thread)

The graphics thread runs completely isolated from all data processing. Every 16.6 milliseconds it reads a read-only snapshot of the atomic global metrics, redraws the dashboard at a perfectly smooth 60 FPS, and returns to sleep — fully decoupled from the capture and parsing pipeline.

5. The Visual User Interface (UI) Specification

The UI is specifically designed to visually demonstrate the **performance gap between sequential and parallel execution**. It features a toggle switch labeled *Execution Mode: [Sequential / Parallel]*.

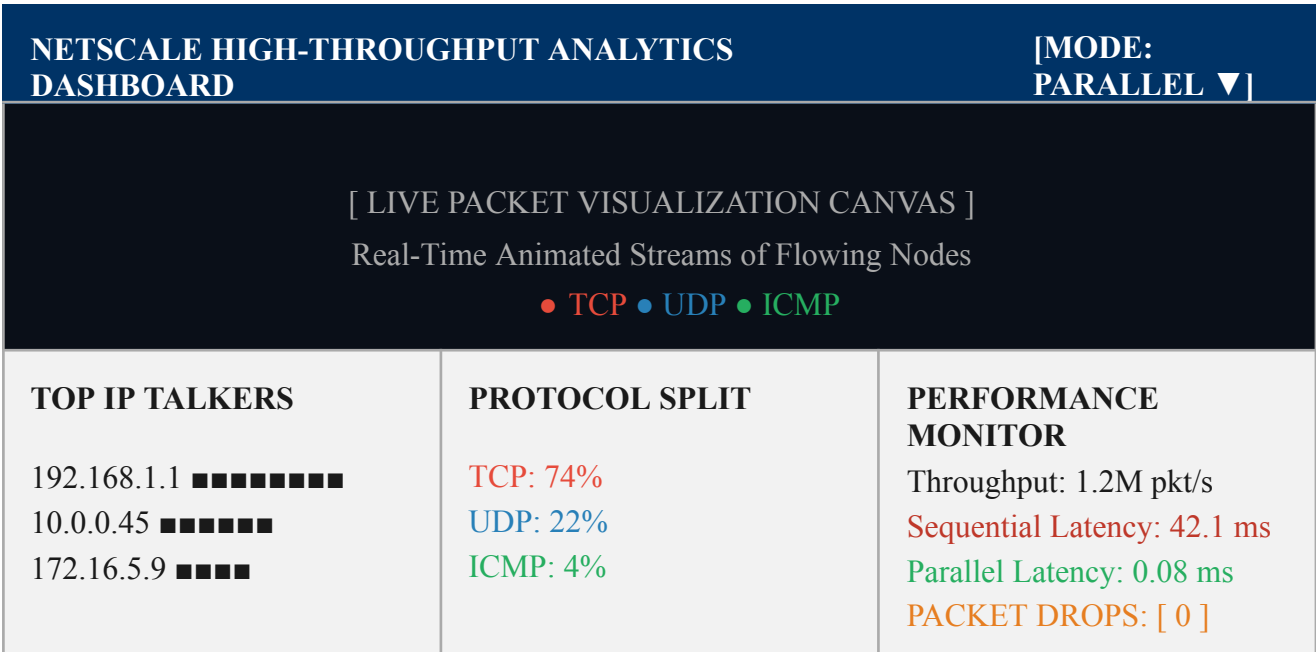


Figure 3 — NetScale Dashboard UI Mockup

Core UI Widgets & Dashboard Elements

- **Live Packet Stream Canvas:** A large central graphical space where processed packets are drawn as animated, moving circles sliding down a time grid. TCP → Red, UDP → Blue, ICMP → Green.
- **Top IP Talkers Chart:** A dynamic, self-sorting bar chart tracking the most active IP addresses, re-sorting in real time as the underlying hash maps are updated.
- **Protocol Distribution Wheel:** A live pie-chart displaying the percentage split between network protocols (TCP vs UDP vs Other).
- **HPC Performance Ticker:** Shows live throughput (Packets/s), latency comparison (Sequential: 30–50 ms vs Parallel: <0.1 ms), and a Packet Drop Counter. In Sequential mode this counter spikes into the thousands; in Parallel mode it stays at a perfect 0

6. Rigorous Complexity & Performance Analysis

The following mathematical analysis quantifies the advantage of the parallel pipeline over naive sequential execution.

Mathematical Notation

Symbol	Definition
N	Total number of packets incoming
P	Number of parallel hardware worker cores
T_parse	Time to perform bitwise parsing of headers and data field lookup
T_render	Time to clear the screen, calculate geometry, and paint pixels
C	Constant rendering cost — fully decoupled from the data pipeline
M	Fixed pre-allocated size of the lock-free circular ring buffer
K	Unique key tracking allocation size for the IP hashing maps

Time Complexity Analysis

Sequential Execution:

$$\text{Time Complexity} = O(N * T_{\text{parse}} + N * T_{\text{render}})$$

Because the thread must sequentially execute both the parsing math and the UI redraw for every loop iteration, the computational load scales **linearly and unmanageably with packet density**. At 1M packets/second, the render overhead alone guarantees packet loss.

Parallel Pipeline Execution:

$$\text{Time Complexity} = O((N * T_{\text{parse}}) / P + C)$$

Packet parsing is distributed across **P worker cores**, so the parsing workload drops linearly with core count. The UI rendering cost C is fully decoupled into an isolated background loop, reducing its overhead on data collection to a flat constant.

Space Complexity

Space Complexity = $O(M + K)$

Where **M** is the fixed, pre-allocated memory size of the lock-free circular ring buffer (allocated at boot to prevent runtime malloc latency), and **K** is the unique key tracking allocation size for the IP hashing maps.

Performance Comparison Summary

Metric	Sequential Mode	Parallel Mode	Improvement
Processing Latency	30 – 50 ms	< 0.1 ms	~500x
Packet Drop Rate	Thousands/second	0	Eliminated
CPU Utilization	1 Core (100%)	N Cores (balanced)	Linear scale-out
UI Responsiveness	Freezes / Lags	Locked 60 FPS	Fully decoupled
Time Complexity	$O(N * T_{\text{parse}} + N * T_{\text{render}})$	$O(N * T_{\text{parse}} / P + C)$	P x speedup
Space Complexity	Unbounded (malloc)	$O(M + K)$ fixed	Deterministic
